

Laboratorio de estructuras concurrentes

Programación Concurrente y Paralela

September 9, 2026

1 Introducción

En este laboratorio vamos a estar trabajando sobre distintas estrategias para implementar una conjunto de valores de tipo `int`.

Este conjunto se implementará sobre una lista enlazada. Para cada ejercicio se espera que se implemente la siguiente interfaz:

```
public interface ConcurrentIntSet {  
    // Agrega un elemento.  
    // Retorna true si este fue agregado, false si ya existe en el conjunto.  
    boolean add(int value);  
  
    // Quita un elemento del conjunto.  
    // Retorna true si el elemento existía, false si no existe el elemento.  
    boolean remove(int value);  
  
    // Retorna true si la lista contiene el elemento.  
    boolean contains(int value);  
  
    // Evita que los ejercicios sin implementar hagan que fallen tests.  
    default boolean isImplemented() { return false; }  
}
```

Los invariantes de la lista siempre deben ser válidos para cualquier combinación de llamadas concurrentes a los métodos públicos.

Los valores de la lista deben ser insertados en orden. Por comodidad para recorrer la lista siempre se inicializa con dos nodos *sentinel* con los valores `Long.MIN_VALUE` y `Long.MAX_VALUE`. Estos acotan todos los valores posibles del conjunto, ya que `Long.MIN_VALUE < Integer.MIN_VALUE < Integer.MAX_VALUE < Long.MAX_VALUE`.

Para resolver este taller pueden usar su IDE preferida que tenga soporte para Maven. Por ejemplo IntelliJ o Visual Studio Code con un plugin de Java + Maven. Alternativamente se provee un makefile para resolver el taller con un editor de texto plano y una versión reciente de JDK instalada.

2 Bloquear toda la lista

La solución más sencilla no siempre es la mejor. La forma más sencilla de permitir que una estructura sea usada de forma concurrente es definir todos los métodos que la utilizan como una zona de exclusión mutua. Completar la clase `CoarseList` para seguir esta estrategia. Se recomienda tomar como referencia la implementación de `SequentialListSet`.

3 Paseando de *lock* en *lock*

Una mejor forma de controlar la concurrencia de una lista es bloquear nodos específicos. El algoritmo básicamente consiste en mantener en todo momento el *lock* del nodo actual y el anterior. Completar la clase `FineListSet`.

Antes de comenzar a escribir código, piensen en las siguientes preguntas:

- ¿Por qué hay que tomar primero el *lock* del predecesor y después el del nodo actual? Piensen qué podría pasar si los locks se toman en cualquier orden.
- Cuando se pasa de un nodo a otro, ¿en qué orden hay que liberar y tomar nuevos *locks*?
- ¿Hay que bloquear nodos en el método `contains`?

4 Sincronización optimista

La idea detrás de este algoritmo es similar al anterior, pero solo tomando *locks* cuando llegamos a la arista de la lista que nos interesa. Sin tomar *locks* en el camino hasta esos nodos. El precio a pagar por hacer esto es tener que verificar que la relación `previo` $\rightarrow$ `actual` siga siendo válida después de tomar los *locks*. Completar la clase `OptimisticListSet`.

Antes de comenzar a escribir código, piensen en las siguientes preguntas:

- ¿Qué hace el método `edgeExists`?
- ¿Qué condiciones hay que asegurar sobre `nodo.next` durante el borrado de un nodo para no romper otros usuarios concurrentes?
- ¿Qué hay que hacer si después de tomar los locks nos damos cuenta que `edgeExists(prev, curr)` es `false`?

5 Sincronización *lazy*

El algoritmo anterior tiene desventajas bastante aparentes. El método `contains` sigue conteniendo *locks* y tenemos que recorrer la lista dos veces para cada operación. Esta es una mejora sobre esas dos cuestiones. Completar la clase `LazyListSet`.

Antes de comenzar a escribir código, piensen en las siguientes preguntas:

- ¿Qué representa el campo `marked` de la clase `Node`?
- Si tanto `marked` como `next` son variables atómicas, ¿por qué sigue haciendo falta un *lock* por nodo?
- ¿Qué se necesita para verificar que la relación `previo` $\rightarrow$ `actual` siga existiendo?

6 Una lista *lock free*

La idea principal detrás de este algoritmo es usar la abstracción `AtomicMarkableReference<T>` para unificar los campos `marked` y `next`.

Para este ejercicio debería alcanzar con estos métodos de la clase `AtomicMarkableReference<Node>`:

```
// Lee solamente la referencia actual.
Node actual = next.getReference();

// Lee la referencia y su marca en una sola operación atómica.
// (La API fea es una limitación de Java de no poder reornar
// múltiples valores fácilmente)
boolean[] marca = new boolean[1];
Node actual = next.get(marca);

// Actualiza ambos valores solo si coinciden los esperados.
// Se ejecuta de forma atómica.
Node referenciaEsperada, referenciaNueva;
boolean marcaEsperada, marcaNueva;
boolean actualizado = next.compareAndSet(
    referenciaEsperada, referenciaNueva, marcaEsperada, marcaNueva);
```

Completar la clase `LockFreeListSet` sin usar ningún *lock*. El uso de la abstracción `Window` es opcional.

Antes de comenzar a escribir código, piensen en las siguientes preguntas:

- ¿Qué hace el método `compareAndSet`? ¿Qué podría ocasionar que este retorne `false`?
- ¿En qué se diferencia conceptualmente el tipo `AtomicMarkableReference<T>` de `volatile T`?

- Analizar el método `Window find(int value)`. Luego explicar el rol que cumple la marca en el campo `next` de la clase `Node`.

7 Experimentación

Para comparar las distintas implementaciones vamos a usar la clase `SetPerformanceBenchmark`. El programa ejecuta varias mezclas de operaciones `contains`, `add` y `remove`, usando una cantidad configurable de hilos y una distribución de claves uniforme o concentrada en una región pequeña de la lista.

Antes de medir se realizan repeticiones de calentamiento para reducir el ruido introducido por el JIT y las cachés internas de la JVM. Para cada configuración se mide la latencia de cada operación, incluyendo los percentiles 90, 95 y 99, y el *throughput* total. Los resultados se muestran por pantalla y también se guardan en un archivo CSV para poder analizarlos después.

- Ejecutar el programa con los parámetros por defecto. ¿Qué resultados se obtienen? ¿Es lo que esperaban ver?
- Cambiar la distribución de generación de claves para ser uniforme. ¿Cómo cambian los resultados en comparación a la corrida anterior?
- Variar el tamaño del espacio de claves, la distribución y en el caso de la clusterizada el tamaño del cluster y la probabilidad de acceder una clave en ese clúster. Ejecutar varios experimentos y comparar resultados.
- Para una serie de parámetros fijos, variar la cantidad de threads. ¿Hasta qué punto aumenta el *throughput*? ¿Tiene alguna relación con la cantidad de CPUs de su computadora?

References

- [1] Maurice Herlihy y Nir Shavit. *The Art of Multiprocessor Programming*, segunda edición. Morgan Kaufmann, 2020. Capítulo 9: “Linked Lists: The Role of Locking”.